



integrata
cegos

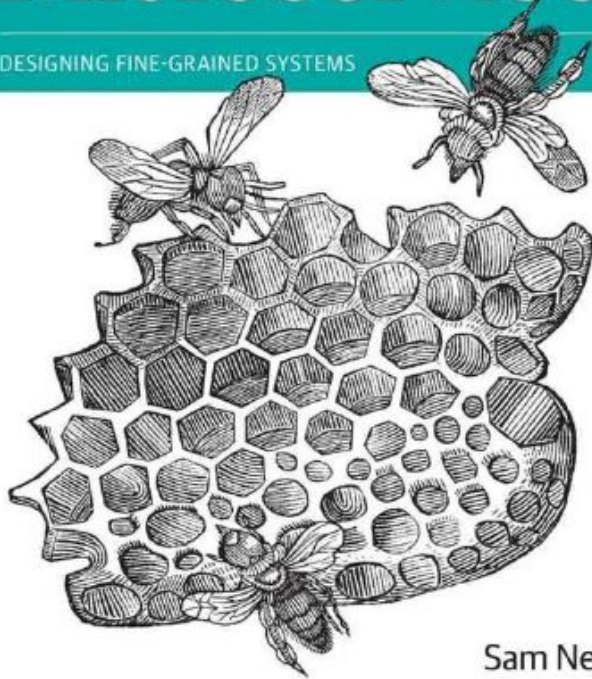
Microservices

Service Oriented Architecture 2.0

O'REILLY®

Building Microservices

DESIGNING FINE-GRAINED SYSTEMS



Sam Newman

MARTINFOWLER.COM

Intro Videos Design Agile Refactoring About Me All Sections   

Microservices

a definition of this new architectural term

The term "Microservice Architecture" has sprung up over the last few years to describe a particular way of designing software applications as suites of independently deployable services. While there is no precise definition of this architectural style, there are certain common characteristics around organization around business capability, automated deployment, intelligence in the endpoints, and decentralized control of languages and data.

- The tools and frameworks used in this seminar are open source
 - LPGL license model
- This is a programming seminar
 - This allows the content to be deepened and internalized through exercises
 - Sample examples are provided
 - These can be copied as a ZIP file at the end of the seminar
 - USB stick or similar
- Documentation and resources are also available on the Internet
- Conventions
 - Commands are displayed in `Courier` font
 - File names are displayed in *Courier font*
 - Links are displayed in Courier font

© Javacream

Javacream

Dr. Rainer Sawitzki

Alois-Gilg-Weg 6

81373 Munich

All rights reserved, including those of excerpts, photomechanical and electronic reproduction.

Introduction	6
Software development	52
Containers for microservices	74

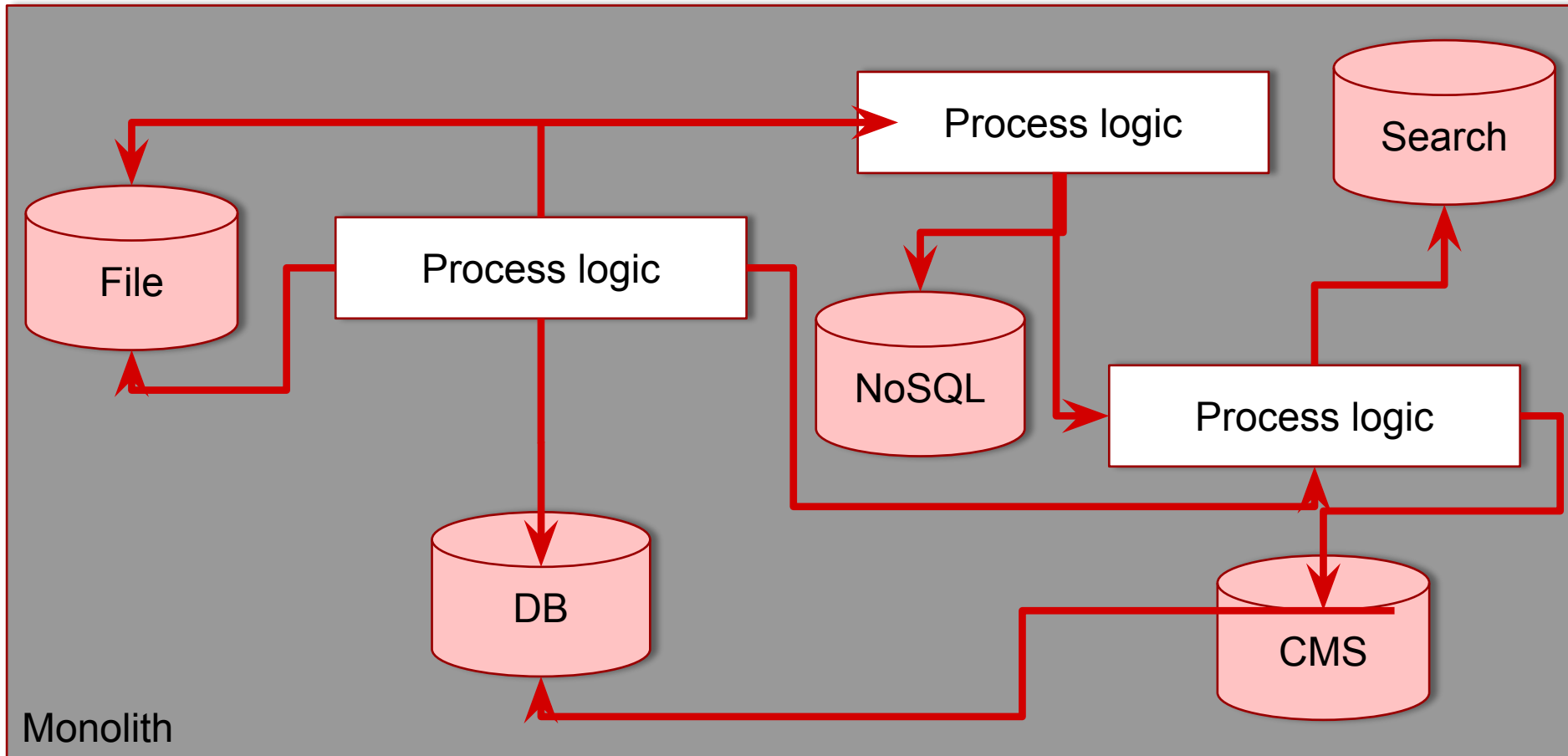
1

INTRODUCTION

1.1

OF MONOLITHS AND SERVICES

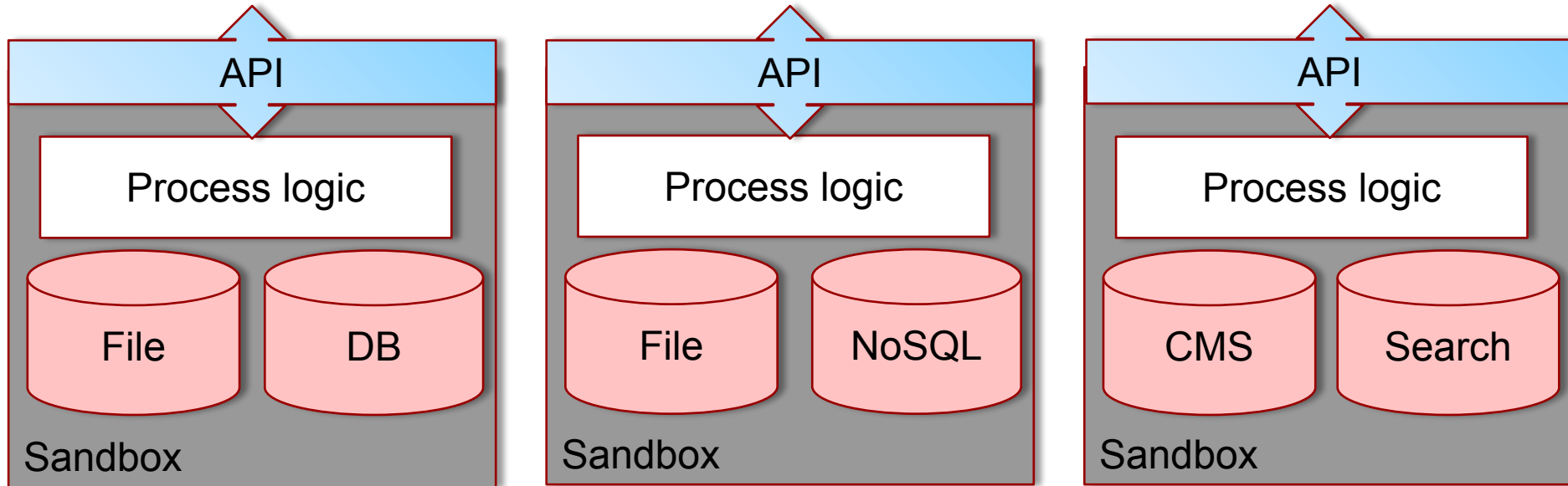
- Primary focus:
 - Application must function professionally
 - Non-functional aspects such as performance and reliability
- Rather uninteresting:
 - Details of the technical modeling
 - Maintainability and expandability, reuse



What would be "better"?

- Encapsulation
 - Public "API"
 - Purely internal implementation
- Avoidance of redundancies
 - Maintainability
- Dependency Management
 - Statically determined dependencies or
 - loosely coupled systems
 - This also makes it easier to expand

Encapsulated services



- Consists abstractly of
 - Data types
 - Operations
 - Synchronous and asynchronous
- Specification with further information
 - Authentication
 - Call via network
 - Non-functional parts
 - Response times
 - Scalability
 - Fault tolerance

- An architecture based on microservices also stands and falls with the documentation
 - Complete
 - so naturally utopian
 - "eventual complete"
 - Current
 - Unambiguous
 - Concise
 - Synchronous realization

- (Formal) prose texts
 - Specialized documentation
- Unified Modeling Language
- Service description with
 - Web Service Description Language
 - WSDL is a specification of the W3W consortium
 - Primarily only in combination with SOAP-based web services
- Swagger
 - Proprietary description language
 - swagger.io
 - UI and code generators for various languages are offered
 - Primarily geared towards RESTful web services
- Elements of a programming language
 - For example, Java interfaces and annotations

- Basic principles of software quality also apply to documentation
 - Don't repeat yourself!
- UML stereotypes
 - Concise name
 - Scope of validity
 - Impact
 - Example:
 - A <<stateless>> system does not build up a conversational memory
- Stereotypes are also known as annotations

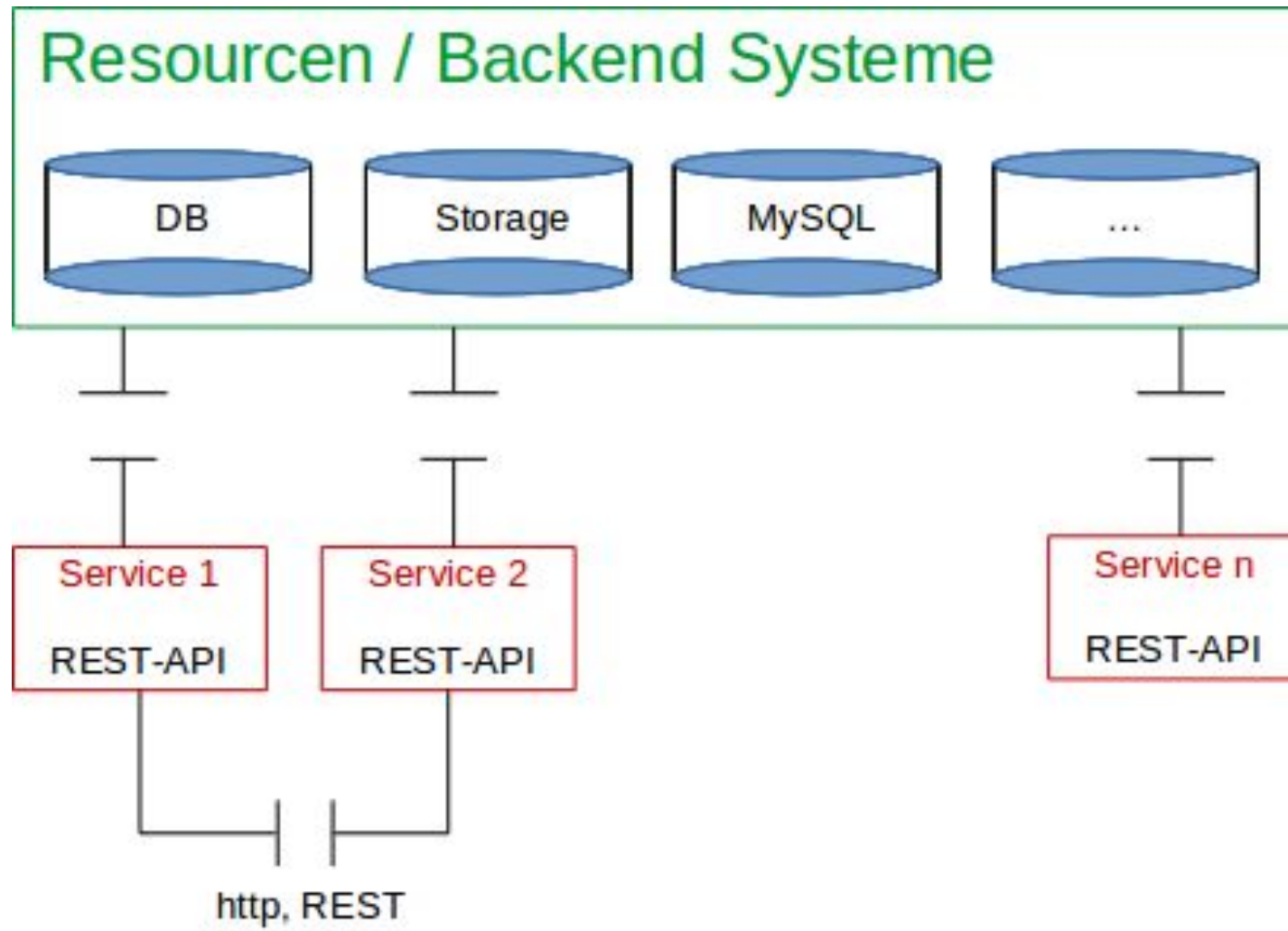
- Established protocols are available
 - SOAP
 - REST
 - Messaging
 - Java RMI
- What is "right"?
 - Messaging system remain
 - The trend is towards REST
 - SOAP is often seen as unnecessarily complex
 - Java RMI is a niche solution

1.2

WHAT ARE MICROSERVICES

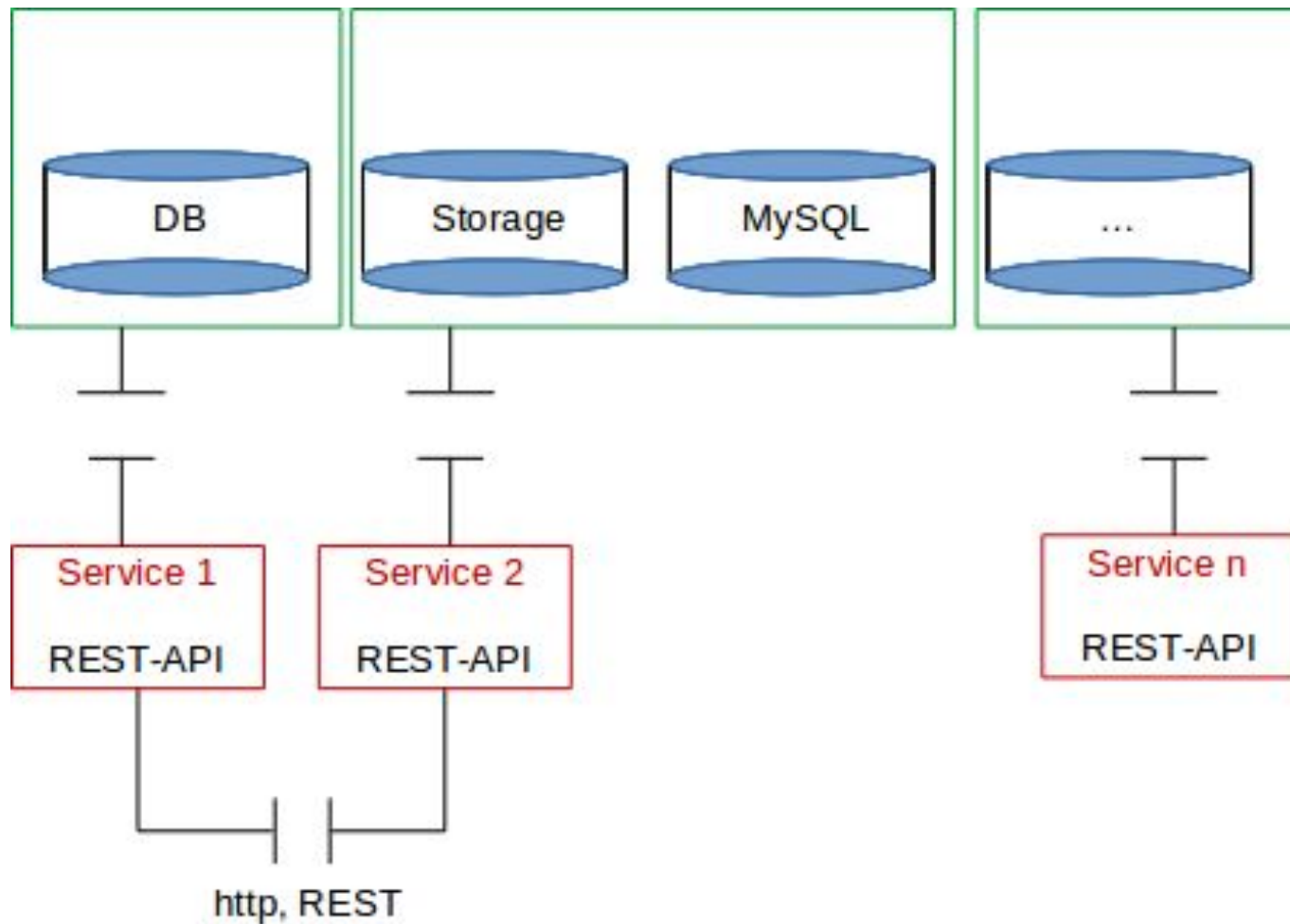
- A complete API description for calling the service is unfortunately not sufficient!
- Services can still remain coupled due to the actually "internal" and encapsulated implementation
 - Shared backend systems!
 - This means that each service has two APIs that need to be described and kept stable

Architecture with shared resources



- WIKI
 - <https://de.wikipedia.org/wiki/Microservices>
- Martin Fowler
 - <https://martinfowler.com/articles/microservices.html>
- Microservice.io
 - <http://microservices.io/patterns/microservices.html>
- Everything is still a bit in flux...

Microservices with encapsulated resources



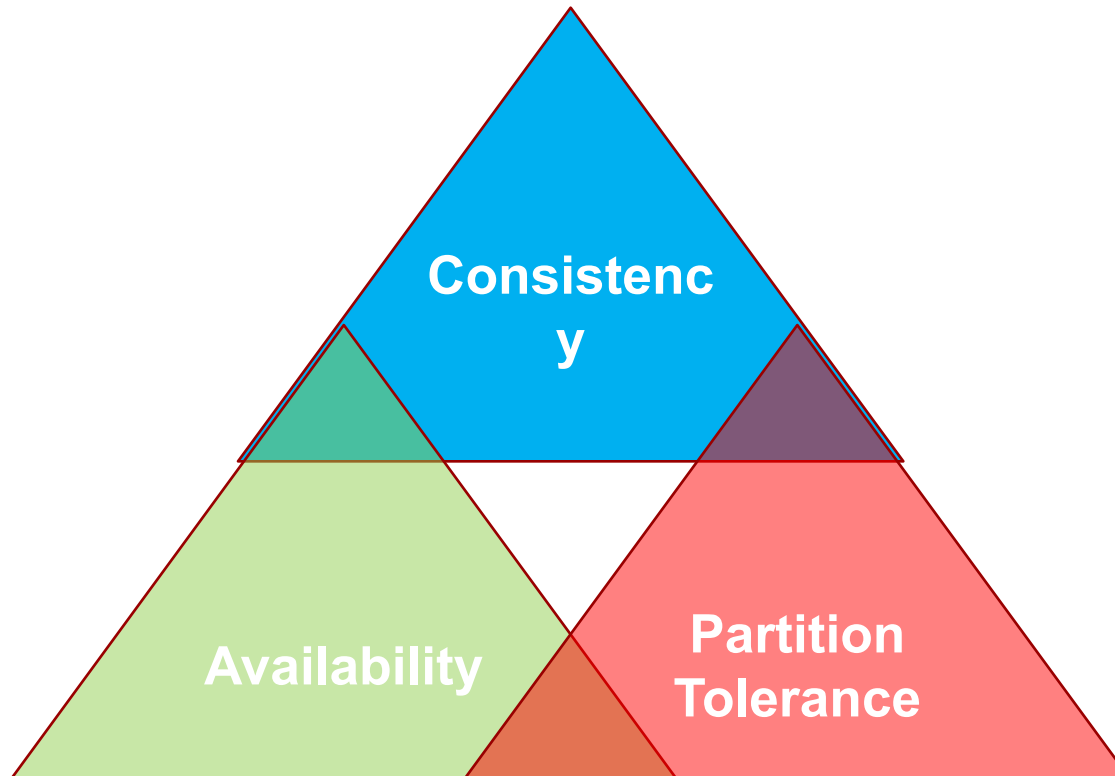
- The microservice is fully described by a versioned API
- The implementation is completely encapsulated
 - Criterion: Even a new implementation is possible without changing other services
- Each microservice brings its own backend resources with it
 - Criterion: Replacement of a backend system is possible

- A microservice
 - "consists of only a manageable number of program lines"
 - "Can be redeveloped in two weeks"
 - "Is the responsibility of small teams"
- A misleading name
 - Even a "small" service can violate the requirements for microservices
 - Even a complex (perhaps even internally monolithically implemented!) application can be operated as a microservice

- Since each microservice encapsulates its resources internally, data redundancies often have to be accepted
 - However, redundancies are often tolerated due to "big & fast data"
 - Microservices therefore have a direct link to NoSQL topics!

- Consistency
 - All nodes have the same level of information at all times
- Availability
 - Every client is guaranteed a response
 - As long as at least one node is running in the cluster
- Partition Tolerance
 - The system tolerates
 - The failure of nodes
 - The failure of the network between nodes

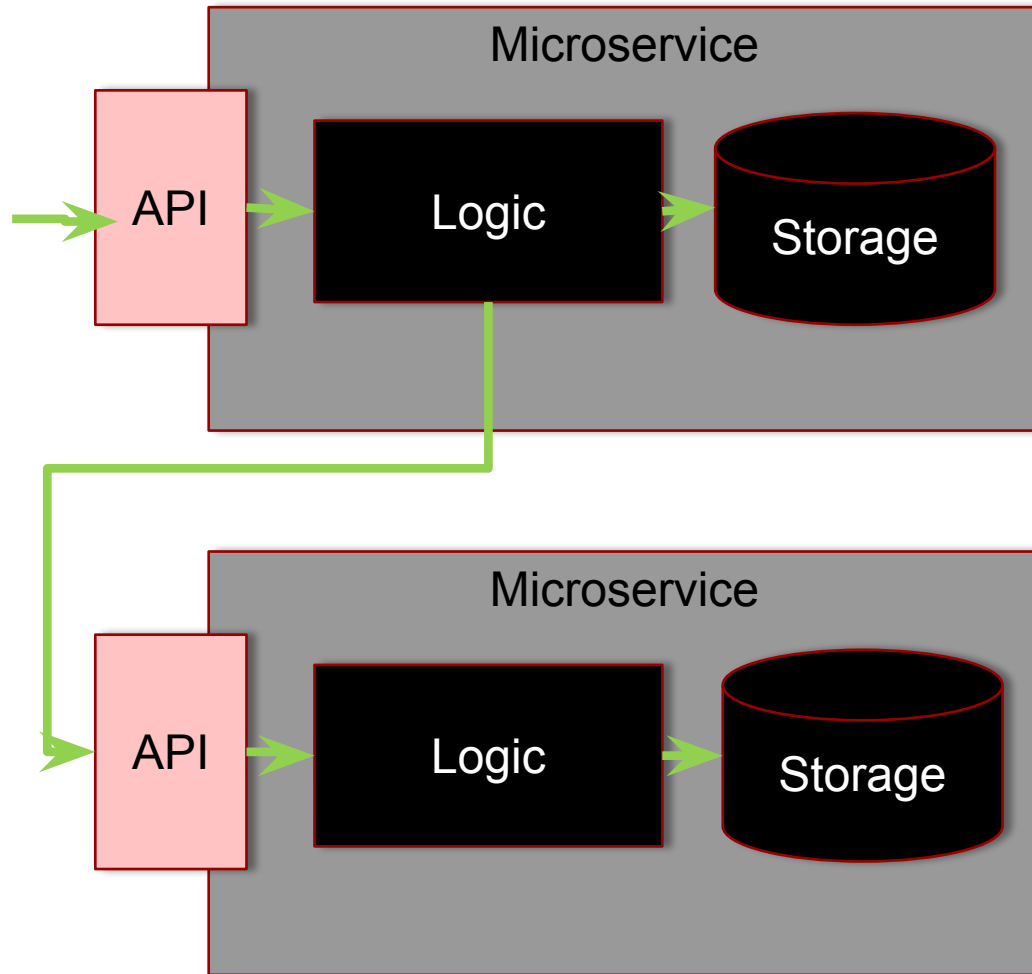
No common intersection!



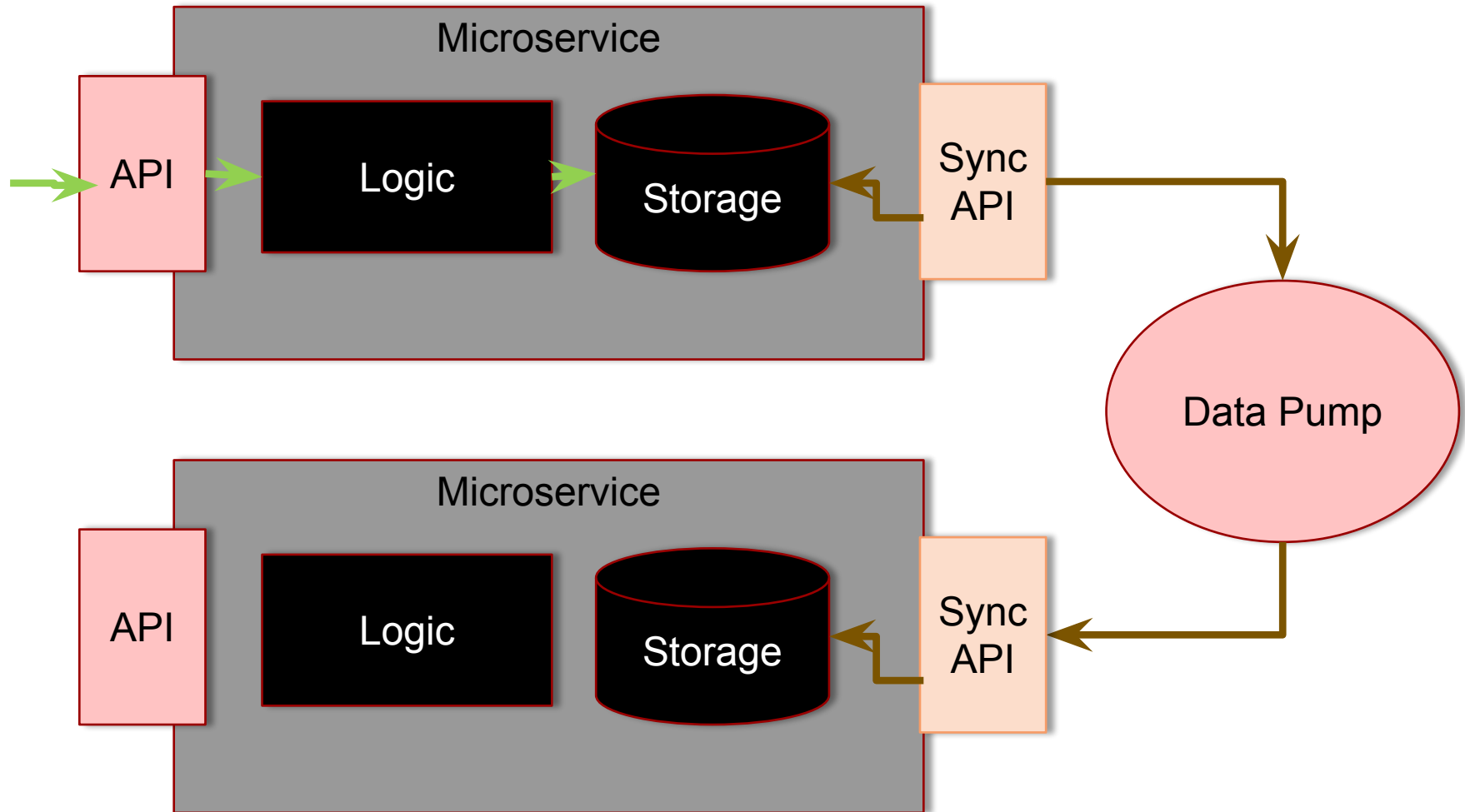
- Consistency
 - All nodes have the same level of information at **all times**
- Eventually Consistent
 - Changes to the database are propagated to the other nodes with a time delay
 - The system ultimately becomes consistent
- BASE
 - Basically Available
 - Soft state
 - Eventual consistency

- ACID
 - Strong consistency of data
 - Transaction management
 - Transaction isolation
 - Two-phase commit if required
 - Relatively complex development
- BASE
 - Weak consistency
 - High availability
 - Speed
 - Fire-and-forget" if required
 - Easier development

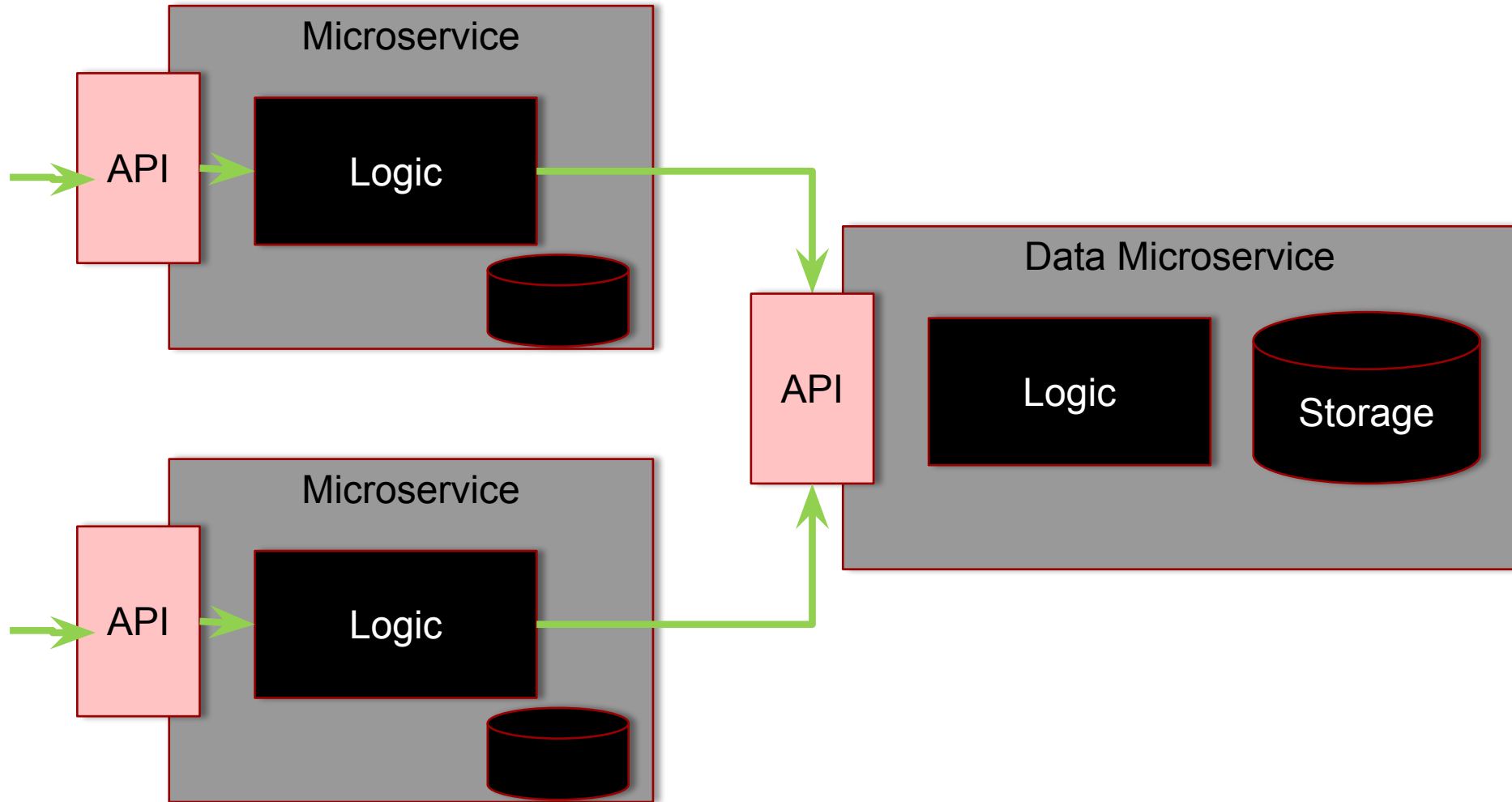
Synchronization through implementation



Synchronization through Data Pump



Synchronization by Data Service



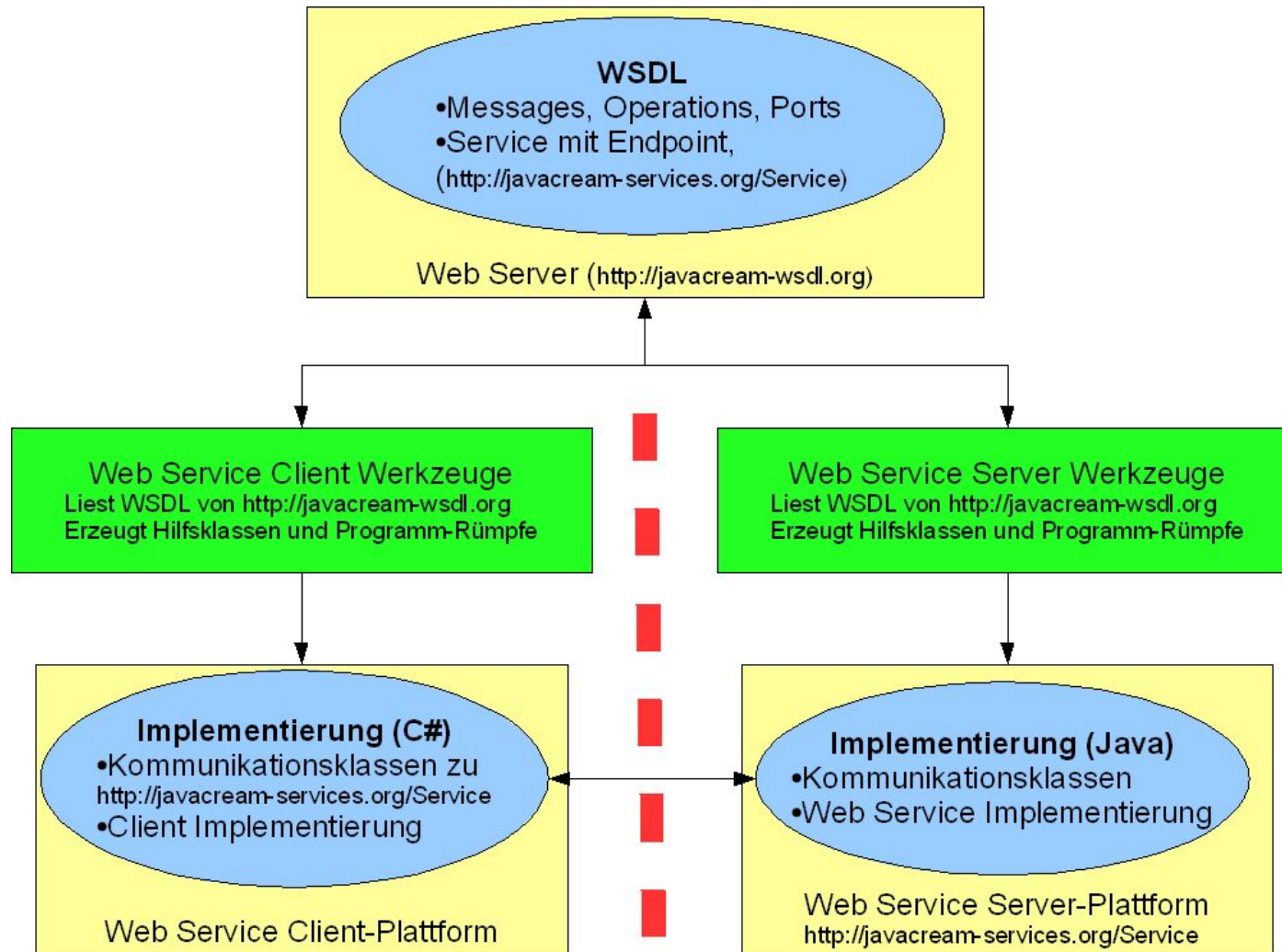
1.3

MODELING

- SOAP-based web services require an interface description
 - Formulation in the Web Services Description Language, WSDL
 - WSDL is an XML schema for describing an interface consisting of
 - Types
 - Operations
 - Ports
 - A WSDL is therefore an XML document
- A SOAP envelope defines the call
 - SOAP is also an XML schema
 - The SOAP document is sent to the destination address of the web service, the endpoint

- A service description is defined in the form of a WSDL
- Realization
 - Code hulls are generated for the server implementation,
 - ready-made stub classes for the client
- When called, the SOAP envelopes are exchanged via the network

Example of a SOAP WebService



The web is not interested in SOAP!

- Not a single browser implementation generates a SOAP envelope for the form submit, for example.
- JavaScript could do this in principle, but there is not a single serious utilities library for it
- System hardware and software are designed for http requests
 - Firewalls
 - Load balancer and router
 - Cache solutions

The Representational State Transfer

- Doctoral thesis by Roy Fielding, 2000
- See <http://en.wikipedia.org/wiki/REST>

Representational state transfer

From Wikipedia, the free encyclopedia
(Redirected from [REST](#))

"REST" redirects here. For other uses, see [Rest](#).

Representational state transfer (REST) is a software architectural style consisting of a coordinated set of architectural constraints applied to components, connectors, and data elements, within a distributed [hypermedia](#) system. REST ignores the details of component implementation and protocol syntax in order to focus on the roles of components, the constraints upon their interaction with other components, and their interpretation of significant data elements.^{[1][2]}

The term *representational state transfer* was introduced and defined in 2000 by [Roy Fielding](#) in his doctoral dissertation at [UC Irvine](#).^{[1][3]} REST has been applied to describe desired web architecture, to identify existing problems, to compare alternative solutions, and to ensure that protocol extensions would not violate the core constraints that make the web successful. Fielding used REST to design [HTTP 1.1](#) and [Uniform Resource Identifiers \(URI\)](#).^{[4][5]} The REST architectural style is also applied to the development of [web services](#)^[6] as an alternative to other distributed-computing specifications such as [SOAP](#).

Contents [\[hide\]](#)

- 1 History
- 2 Software architecture
 - 2.1 Components
 - 2.2 Connectors
 - 2.3 Data
- 3 Architectural properties
- 4 Architectural constraints
 - 4.1 Client-server
 - 4.2 Stateless
 - 4.3 Cacheable
 - 4.4 Layered system
 - 4.5 Code on demand (optional)
 - 4.6 Uniform interface

- The way the Internet works abstracted
 - What are resources?
 - How are resources identified?
 - What are resource operations?
 - How are services described?
 - What are stateless operations?
 - Requirements and implementation of caching mechanisms
 - Optional: Transfer of script logic to the client
- The basic concept is the linked HyperText documents
- Not surprisingly: "The Internet is an example of the implementation of a REST-based system"

The http protocol

- A comprehensive specification of the w3w consortium
 - See <http://en.wikipedia.org/wiki/Http>

Hypertext Transfer Protocol

From Wikipedia, the free encyclopedia
(Redirected from [Http](#))

The **Hypertext Transfer Protocol (HTTP)** is an [application protocol](#) for distributed, collaborative, [hypermedia](#) information systems.^[1] HTTP is the foundation of data communication for the [World Wide Web](#).

[Hypertext](#) is structured text that uses logical links ([hyperlinks](#)) between [nodes](#) containing text. HTTP is the protocol to exchange or transfer hypertext.

The standards development of HTTP was coordinated by the [Internet Engineering Task Force](#) (IETF) and the [World Wide Web Consortium](#) (W3C), culminating in the publication of a series of [Requests for Comments](#) (RFCs), most notably [RFC 2616](#) [\[4\]](#) (June 1999), which defines HTTP/1.1, the version of HTTP in common use.

Contents [\[hide\]](#)

- 1 Technical overview
- 2 History
- 3 HTTP session
- 4 Request methods
 - 4.1 Safe methods
 - 4.2 Idempotent methods and web applications
 - 4.3 Security
- 5 Status codes
- 6 Persistent connections
- 7 HTTP session state
- 8 Encrypted connections
- 9 Request message
- 10 Response message
- 11 Example session
 - 11.1 Client request

Internet protocol suite

Application layer

BGP • DHCP (DHCPv6) • DNS • FTP •
HTTP • IMAP • IRC • LDAP • MGCP •
NNTP • NTP • POP • RPC • RTP • RTSP •
RIP • SIP • SMTP • SNMP • SOCKS • SSH •
Telnet • TLS/SSL • XMPP • *more...*

Transport layer

TCP • UDP • DCCP • SCTP • RSVP •
more...

Internet layer

IP (IPv4 • IPv6) • ICMP • ICMPv6 • ECN •
IGMP • IPsec • *more...*

Link layer

ARP/InARP • NDP • OSPF • Tunnels (L2TP)
• PPP • Media access control (Ethernet •
DSL • ISDN • FDDI • DOCSIS) • *more...*

V • T • E

- Definition of URIs
 - Path
 - Parameters
- http request and http response
 - Data container with header and body
 - Encoding
- Comprehensive set of header properties
 - Content-Length
 - Accepts
 - Content-Type

- http methods
 - PUT
 - GET
 - POST
 - DELETE
 - OPTIONS
 - HEAD
- Status codes for calls
 - 404: "Not found"
 - 204: "Created"
 - ...

- Definition of the data types of the Internet
 - Not to be confused with an XML schema
 - A MimeType is "only" a structured character string
 - Own extensions are possible

- REST has nothing to do with http in principle
 - REST is an abstract architecture
 - http is a concrete communication protocol
- But
 - As the communication protocol of the "reference implementation" Internet, http naturally fits perfectly with the REST style

- http methods and resource operations
 - PUT
 - Creating a new resource
 - Update
 - GET
 - Reading a resource
 - POST
 - Update
 - New plant
 - DELETE
 - Delete
- Idempotence
 - Idempotent operations may be called any number of times and do not cause any side effects
 - REST requires an idempotent implementation for PUT and DELETE

- With PUT
 - The client must also specify the resource ID
 - Return is a status code "201: Created"
- With POST
 - The server decides whether it needs to create a new resource
 - If yes:
 - Status code "201: Created"
 - Set `location` header with URI of the resource just created
 - Optional: Body contains the created resource

Conception of a RESTful service: Update

- With PUT
 - Status code "200: OK" or "204: No content"
 - PUT is idempotent (!)
- With POST
 - POST is used for non-idempotent updates

- With DELETE
 - Status code "200: OK" or "204: No content"
 - PUT is idempotent (!)
- Conceptually, a distinction must be made:
 - A "real" DELETE deletes the resource
 - A functional deletion (e.g. cancellation) is actually an update of the resource
 - However, overloading the http-DELETE is perfectly legitimate for these purposes
 - `DELETE order/ISBN42?cancel=true`

- As a first approximation, a RESTful web service does not require a separate description
 - PUT, GET, POST, DELETE cover all relevant resource operations
 - Data types are determined by MIME types
- The only information required is the endpoint address
- In practice, however, additional information is required in most cases
 - For example, a GET to the endpoint returns a list of all resources
 - But:
 - List of all information or references?
 - Determining the sorting sequence?
 - ...

- Analogous to the WSDL, a more detailed service description is provided
 - Web Application Description Language (WADL)
 - http://en.wikipedia.org/wiki/Web_Application_Description_Language
 - Rest Service Description Language (RSDL)
 - <http://en.wikipedia.org/wiki/RSDL>
 - Swagger
 - <http://swagger.io>

A completely different approach: HATEOAS

- Hypertext as the Engine of Application State
 - <http://en.wikipedia.org/wiki/HATEOAS>
- Basic idea: Each response contains a set of links that define the further meaningful actions
 - "Sounds like a web application!"
 - But now consistently with PUT, DELETE

How big is "Micro"?

- To be found in the literature: "As small as possible"
 - So naturally nonsensical
 - The smallest possible microservice
 - contains exactly one operation
 - returns a primitive result
- Better: "As big as necessary"
 - Criteria:
 - Data replication effort across different microservices
 - Transaction contexts
 - Performance aspects

2

SOFTWARE DEVELOPMENT

2.1

JAVA, JEE AND SPRING BOOT

- The Java Virtual Machine is an encapsulated runtime environment for JAva applications
- Parallel execution on one hardware possible without any problems
- Connection to system monitoring is provided

- The Java Enterprise Edition (JEE) extends the Java Virtual Machine with additional infrastructure
 - Application server
- The application server executes JEE applications
 - Encapsulation within the application server
 - Hot Deployment
 - Inter-app communication
 - In memory through linking
 - Internal bus system
 - Messaging in particular
 - Support for all common communication protocols

- A widely used open source framework as an alternative to the JEE application server
- Spring Core component and many associated projects
 - Spring Data
 - Spring MVC
 - Spring Integration
 - ...

- A Spring Boot application is delivered as a single deployment archive
 - and started via a simple Java call
- This makes Spring Boot an excellent fit for Docker
 - A simple Java image serves as the basis
 - The application to be installed is added as a single file
 - and started via a standard Java call

2.2

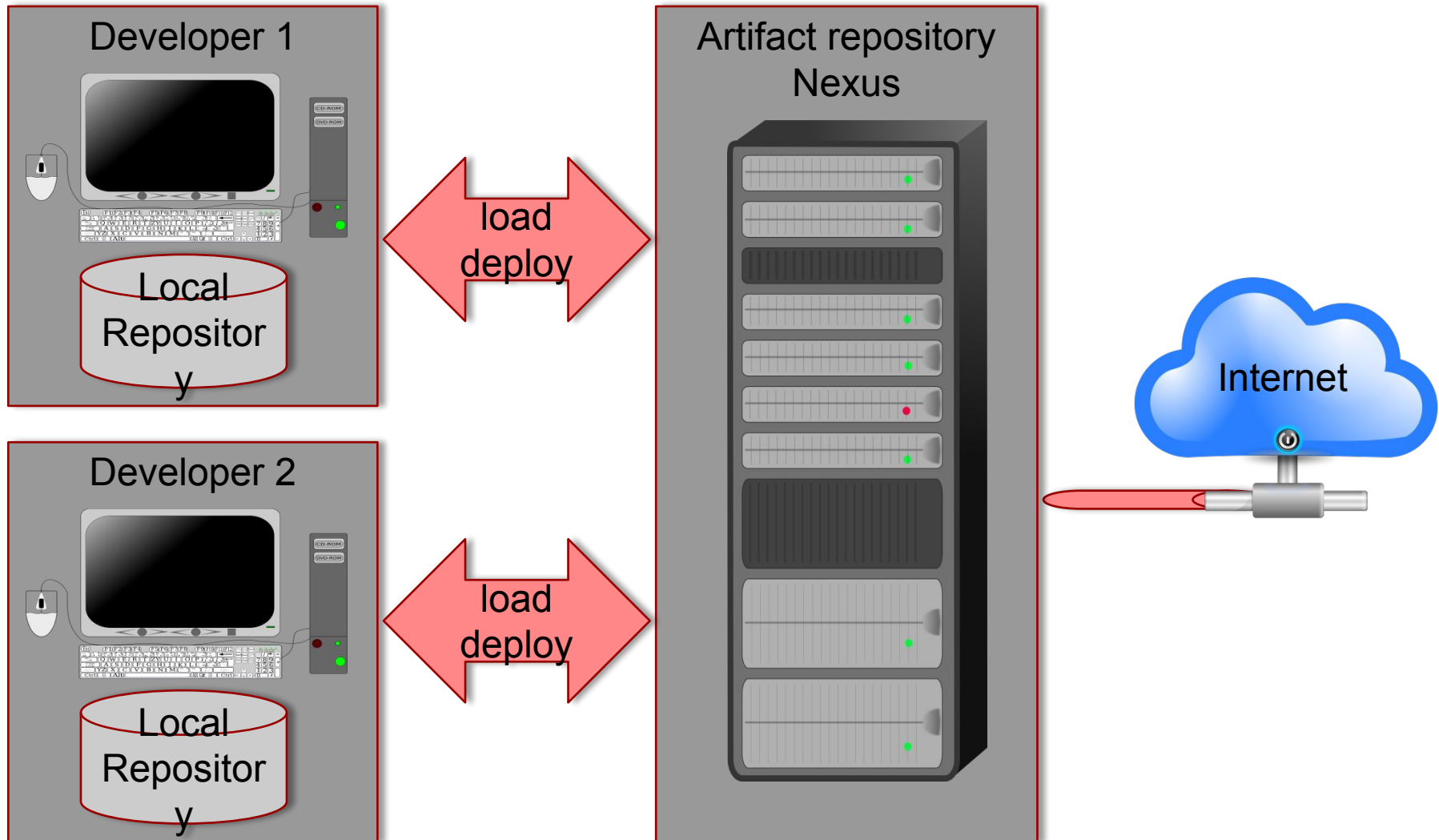
BUILD PROCESS

- Reproducible
- Cross-platform
 - Developer calculator
 - Build-Machine
- Creates "artifacts"

- Source code management
 - Contains the source codes, configurations of the application
 - the so-called "works"
 - The build processes are also works
- Build machine
 - Contains the build jobs and executes them
- Artifact repository
 - The results generated by the build process are called artifacts
 - These are kept in a versioned structure in a repository

- Widely used in Java-based software development
- Components
 - Fixed build process
 - Call with mvn <command>
 - compile
 - package
 - install
 - deploy
- Local cache for created artifacts
- Connectivity to a remote repository
 - Standard: Internet repository repo1.maven.org
 - In the company: Own repository server
 - Nexus
 - Artefactory
 - ...

- Build sequence
 - Compiler call
 - JAR archiver
 - + x, e.g. check-out from version management, testing, deploying from servers, etc.
- Dependency Management
 - Other internal software projects
 - Open source projects
 - Product libraries



- **settings.xml, generally valid**

```
<settings>
```

```
  <mirrors>
```

```
    <mirror>
```

The address of the internal server is entered here

```
    </mirror>
```

```
  </mirrors>
```

```
<servers>
```

```
  <server>
```

Authentication information for the internal server

```
  </server>
```

```
</servers>
```

```
</settings>
```


- This is done by a so-called parent POM
 - A kind of superclass for all Maven build processes
 - This is stored as a separate artifact in the repository
- Example:

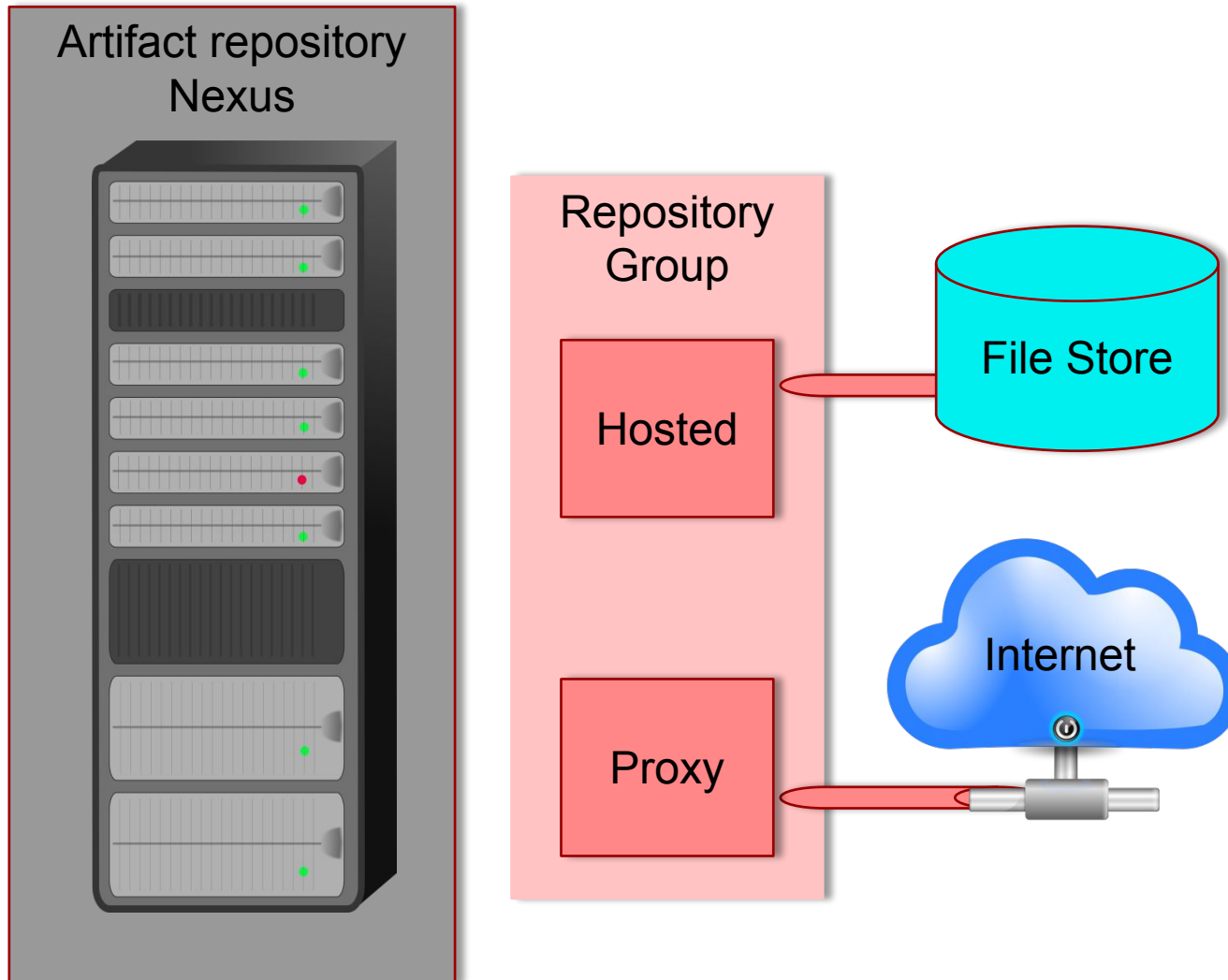
- Extract from distribution management

```
<distributionManagement>
  <repository>
    <uniqueVersion>false</uniqueVersion>
    <id>nexus</id>
    <name>Corporate Repository</name>
    <url>http://...</url>
  </repository>
  <snapshotRepository>
    <uniqueVersion>true</uniqueVersion>
    <id>nexus</id>
    <name>Corporate snapshots</name>
    <url>http://...</url>
  </snapshotRepository>
</distributionManagement>
```

- Configuration of the Java compiler
 - Java version
- Reporting
- Dependencies for other artifacts for general use
 - JUnit
 - Database driver
 - ...
- Configuration of further plug-ins
 - Special reports

- Ready-made product solution from Sonatype
 - Community
 - Commercial support
- Repository server for
 - Java artifacts
 - But also Docker images
- Alternative products
 - Apache Archiva
 - JFrog Artefactory
 - Atlassian Bitbucket

Basic organization



- The repository group is used for read operations
 - Non-existent artifacts or images are reloaded via the Internet proxy
- The hosted repository is used to store your own artifacts and images

Nexus Maven repository

Maven - Nexus Repository Manager - Mozilla Firefox

Maven - Nexus Rep... x +

localhost:8081/#browse/search/maven

Nexus Repository Manager OSS 3.2.1-01

Search components

admin Sign out

Browse

- Welcome
- Search
 - Custom
 - Docker
 - Maven
 - NuGet
- Browse
 - Assets
 - Components

Maven Search for components by Maven coordinates

Group Id: Any Artifact Id: Any Version: Any Base Version: Any Classifier: Any Extension: Any

More criteria

Only showing the first 1,000 of 1,145 results

Name ↑	Group	Version	Format
org.eclipse.sisu.plexus	org.eclipse.sisu	0.3.0	maven2
org.javacream.training.application.book	org.javacream.training.application.book	0.0.1-20170310....	maven2
org.javacream.training.application.book	org.javacream.training.application.book	0.0.1-20170310.1...	maven2
org.javacream.training.application.book.api	org.javacream.training.application.book	0.0.1-20170305...	maven2
org.javacream.training.application.book.api	org.javacream.training.application.book	0.0.1-20170310....	maven2
org.javacream.training.application.book.api	org.javacream.training.application.book	0.0.1-20170310.1...	maven2
org.javacream.training.application.book.cdi	org.javacream.training.application.book.cdi	0.0.1-20170310....	maven2
org.javacream.training.application.book.cdi	org.javacream.training.application.book.cdi	0.0.1-20170310.1...	maven2
org.javacream.training.application.book.cdi.jee	org.javacream.training.application.book.cdi	0.0.1-20170310....	maven2
org.javacream.training.application.book.cdi.jee	org.javacream.training.application.book.cdi	0.0.1-20170310.1...	maven2
org.javacream.training.application.book.cdi.jee.jp	org.javacream.training.application.book.cdi	0.0.1-20170310....	maven2
org.javacream.training.application.book.cdi.jee.jp	org.javacream.training.application.book.cdi	0.0.1-20170310.1...	maven2
org.javacream.training.application.book.cdi.jee.memory	org.javacream.training.application.book.cdi	0.0.1-20170310....	maven2
org.javacream.training.application.book.cdi.jee.memory	org.javacream.training.application.book.cdi	0.0.1-20170310.1...	maven2
org.javacream.training.application.book.cdi.spring	org.javacream.training.application.book.cdi	0.0.1-20170310....	maven2
org.javacream.training.application.book.cdi.spring	org.javacream.training.application.book.cdi	0.0.1-20170310.1...	maven2

Copyright © 2008-present, Sonatype Inc. All rights reserved.

2.3

INTEGRATION AND MONITORING

- Monitoring close to the operating system is not particularly informative for Java processes
 - Metrics of the Java Virtual Machine
 - Heap memory
 - Garbage Collections
- JMX is better used for this
 - Idea: JMX is provided on a (company-standardized) port, for example with Jolokia
 - However, there are also other solutions for this



[Home](#)
[Download](#)
[Features](#)
[Documentation](#)
[Support](#)
[Blog](#)
[About](#)

Jolokia

Download

Features

Support

Forum

IRC

License

Documentation

Overview

Tutorial

Reference Manual

Talks and Screenshots

Agents

Overview

Web Archive (war)

Osgi

JVM

Mule

Jolokia is remote JMX with JSON over HTTP.

It is fast, simple, polyglot and has unique features. It's JMX on Capsaicin.

Jolokia is a JMX-HTTP bridge giving an alternative to JSR-160 connectors. It is an agent based approach with support for many platforms. In addition to basic JMX operations it enhances JMX remoting with unique features like bulk requests and fine grained security policies.

Starting points

- Overview of **features** which make Jolokia unique for JMX remoting.
- The **documentation** includes a **tutorial** and a **reference manual**.
- Agents** exist for many platforms (JEE, OSGi, Mule, JVM).
- Support** is available through various channels.
- Contributions** are highly appreciated, too.

News

3

CONTAINERS FOR MICROSERVICES

3.1

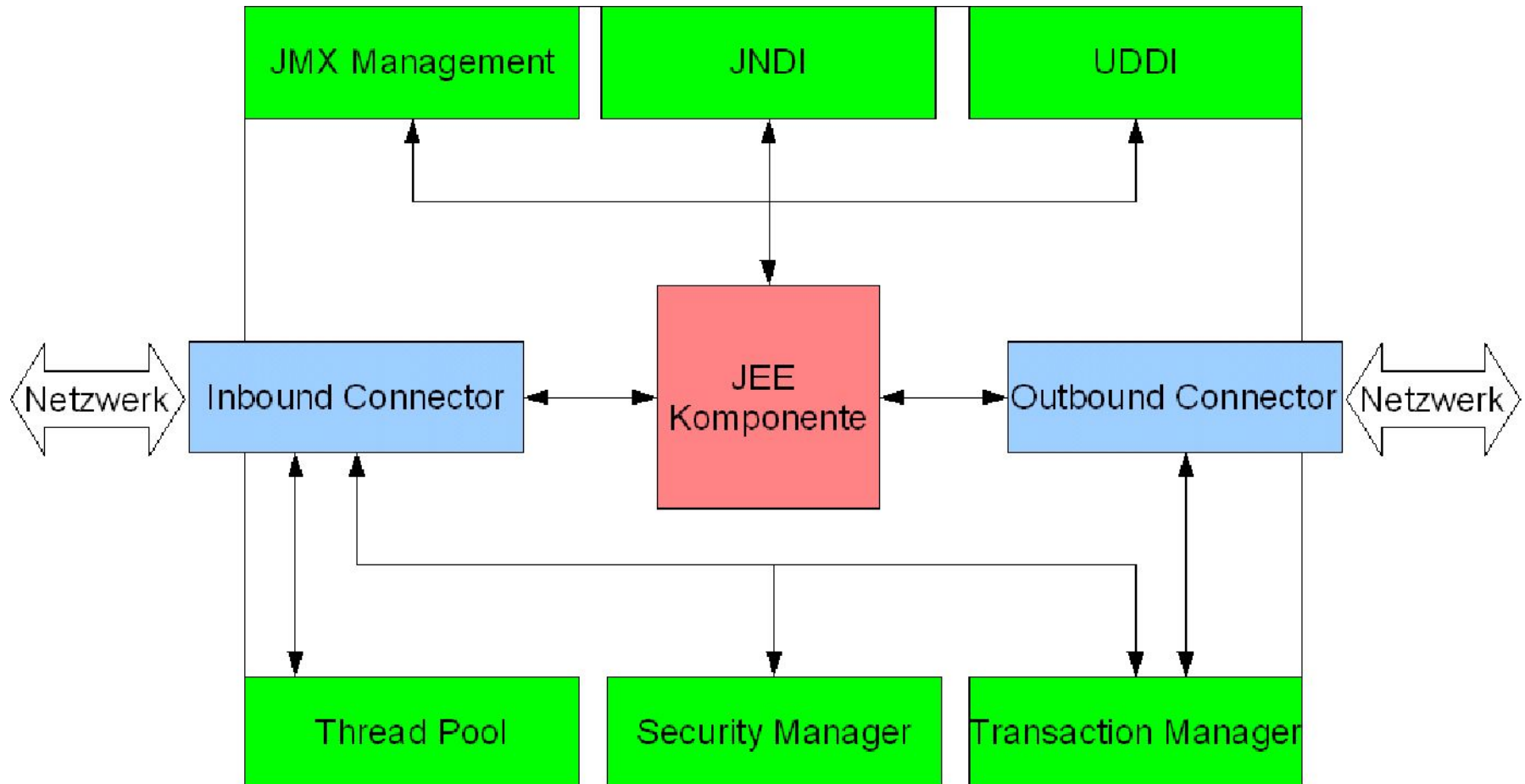
APPLICATION SERVER

- Application servers are part of the Java Enterprise Edition
 - A specification
- Characteristics:
 - Web Profile
 - Web applications
 - Web Services
 - Full Profile
 - Transactional resource access
 - Messaging
 - Other unofficial profiles
 - Pure messaging systems
 - Web profiles with simple transaction management

- Deploying applications to the running server
 - Hot Deployment
 - File transfer to a monitored directory
 - Administrative process
 - Web Console
 - Admin scripts
 - Configurative
 - with restart of the server

- The server is accessible via JMX
- Web consoles
- Log files
- Operating system monitoring
 - Memory
 - CPU
 - IO
- When using the Docker container, it takes on the role of the operating system!

The application server



- Inbound connectors enable communication with the application server
 - http/https
 - Java Remote Method Invocation (RMI) or the interoperable variant RMI/IIOP
 - Receiving messages with the Java Messaging Service (JMS)
 - SOAP
- For protocols that are not included in the standard, special implementations for Inbound Connectors can be installed as plug-ins in the application server
 - There is also a broad market of manufacturers and products for this purpose

- The inbound connectors themselves do not contain any business processes
- By design, they convert an incoming network connection into a request which, as mentioned above, is executed by a thread from the thread pool
- Business logic is defined in a JEE component
- An inbound connector can be called up as part of a distributed transaction

- JEE components consist of two components:
 - A container provided by the application server
 - and the actual component, which is to be understood as part of an application program

- Servlets are called by the http Inbound Connector
 - The decision as to which servlet (and therefore which application logic) is to be called is made via the URL of the http call
 - A special type of servlet can also be called via SOAP
 - In modern applications, there is probably only one central servlet
- Enterprise JavaBeans are primarily used for transaction control
 - The application programmer is offered various sub-types that differ in terms of life cycle and call protocol
 - There are "Stateless" and "Stateful SessionBeans", "Message Driven Beans" and "Singletons"
 - Enterprise JavaBeans are often no longer found in modern applications

- Outbound Connectors are used locally by JEE components
- Your task is to call up a business process within a backend system
 - The Outbound Connector holds a connection pool for this purpose
- The Outbound Connectors are also used by the Transaction Manager (for the coordination of distributed transactions) and the Security Manager (propagation of the logged-in user)

- The JEE specification requires the following outbound connectors:
- The data sources connect to SQL database systems
- Sending JMS messages
- Connecting to an email system
- All other systems can be installed using plug-ins, for which there is a broad market

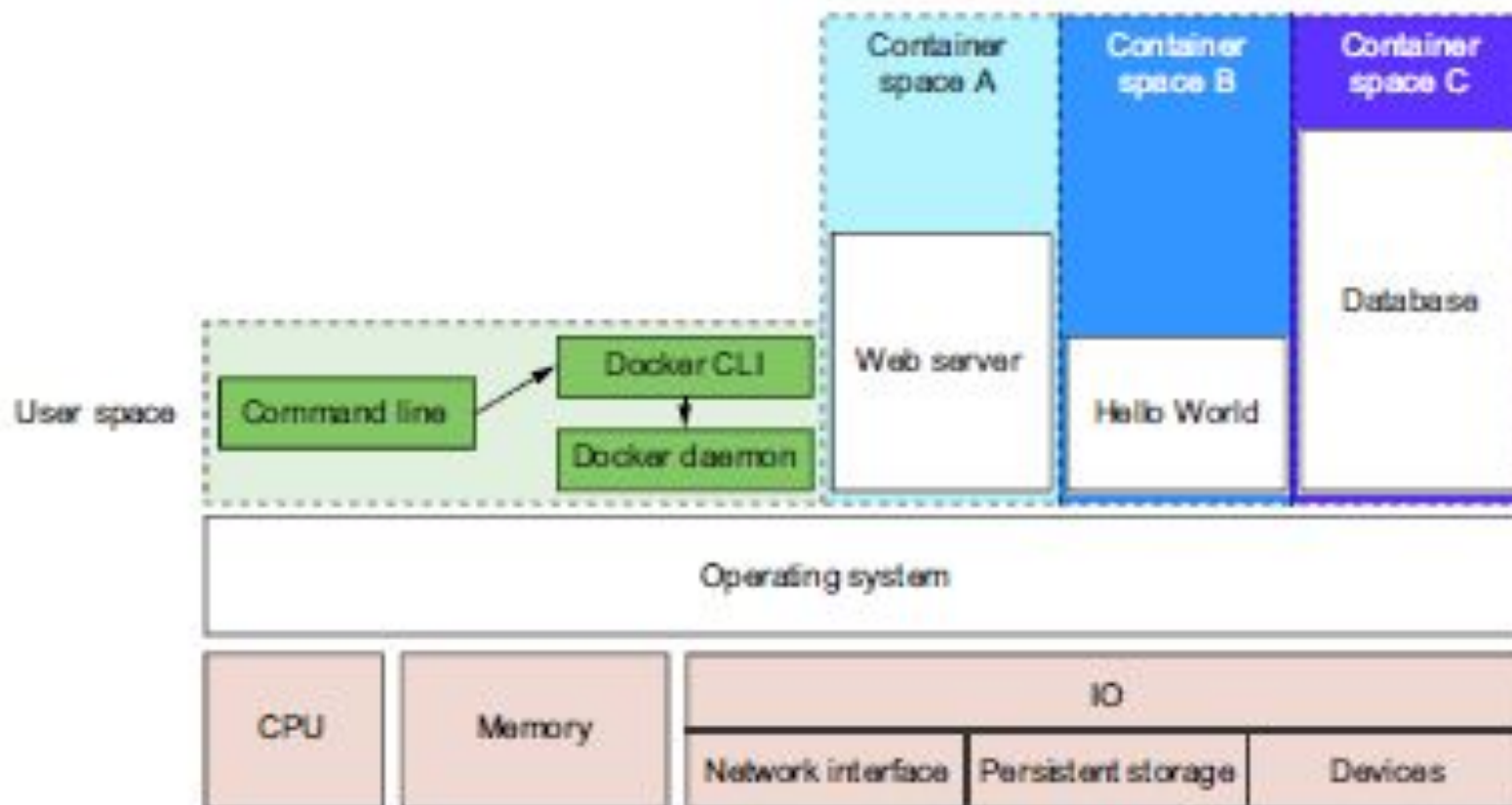
3.2

DOCKER

What is a container?

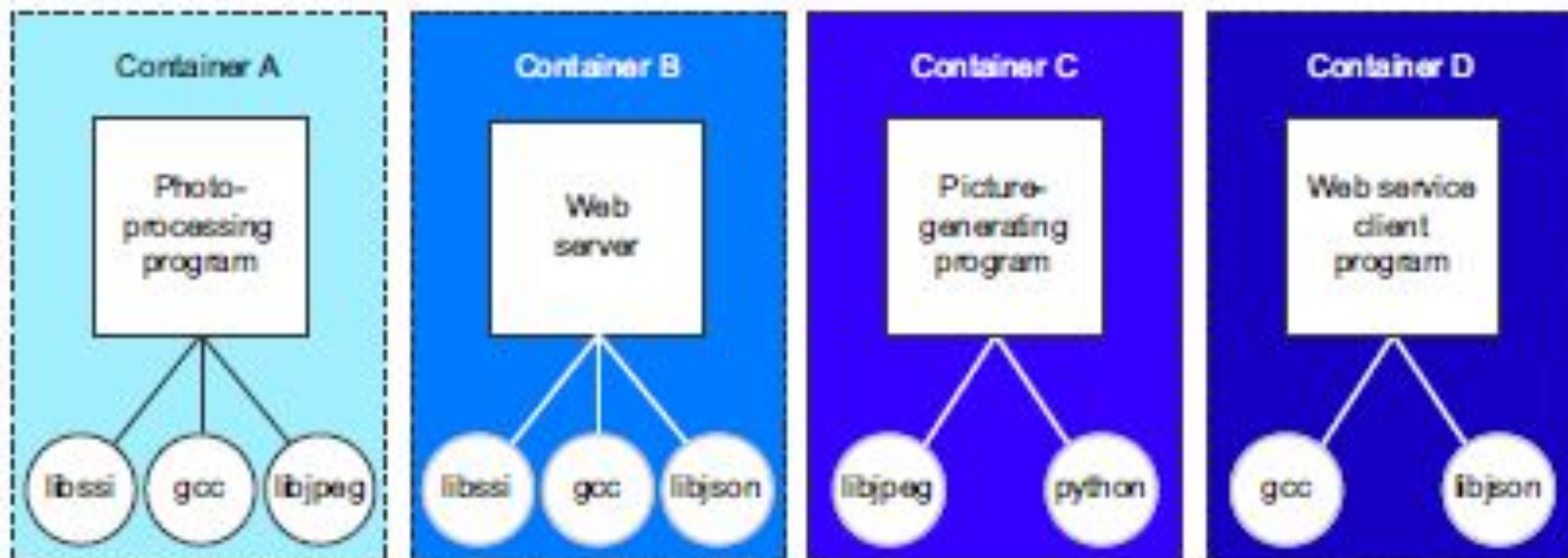
- "A container is a (modified) runtime environment that prevents the application running in it from accessing protected resources."
 - Difference to virtualization: Here, a complete hardware is abstracted by a "virtual machine"
- Docker containers run directly on the Linux kernel without further emulation
 - established Linux features such as namespaces and cgroups are used

Container insulation



- Docker Command Line
 - This command is used directly by the Docker user
- Docker host
 - A machine with an installed Docker environment
 - Docker daemon/engine
 - Local registry
- Docker daemon/engine
 - The Docker containers are started via the daemon
 - Technically speaking, the container child processes of the daemon are
- Remote Services
 - Registries for Docker images
 - DockerHub

- Microservices
 - Well-defined API
 - not relevant here
 - Network
 - Isolation of the internal implementation
 - Provision of all required resources
 - Internal resources such as databases
 - Dependencies on other services
- Docker fits!



- After installation, the first container can be started (!)
 - `docker run <docker_image>`
 - e.g. `docker run hello-world`
- That's it

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
535020c3e8ad: Pull complete
af340544ed62: Pull complete
Digest: sha256:a68868bfe696c00866942e8f5ca39e3e31b79c1e50feaaee4ce5e28df2f051d5c
Status: Downloaded newer image for hello-world:latest
Hello from Docker.

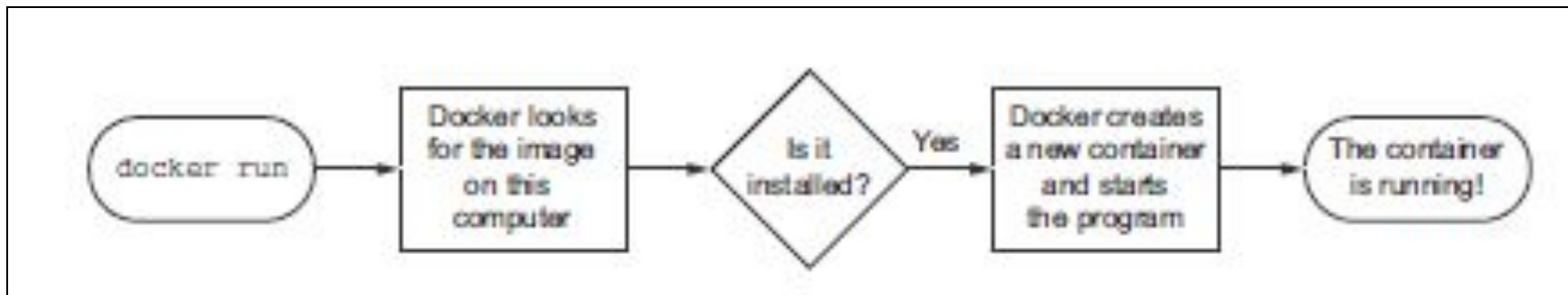
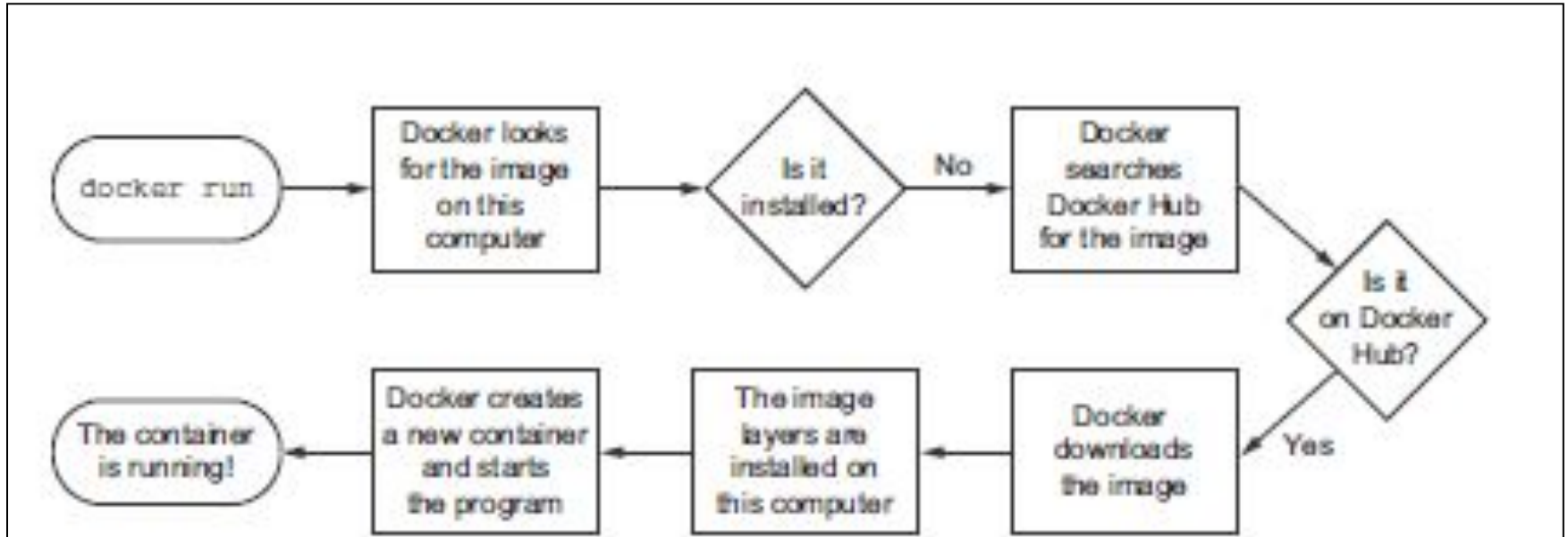
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. the Docker Engine CLI client contacted the Docker Engine daemon.
2. the Docker Engine daemon pulled the "hello-world" image from the Docker Hub.
3. the Docker Engine daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. the Docker Engine daemon streamed that output to the Docker Engine CLI client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker Hub account:
https://hub.docker.com

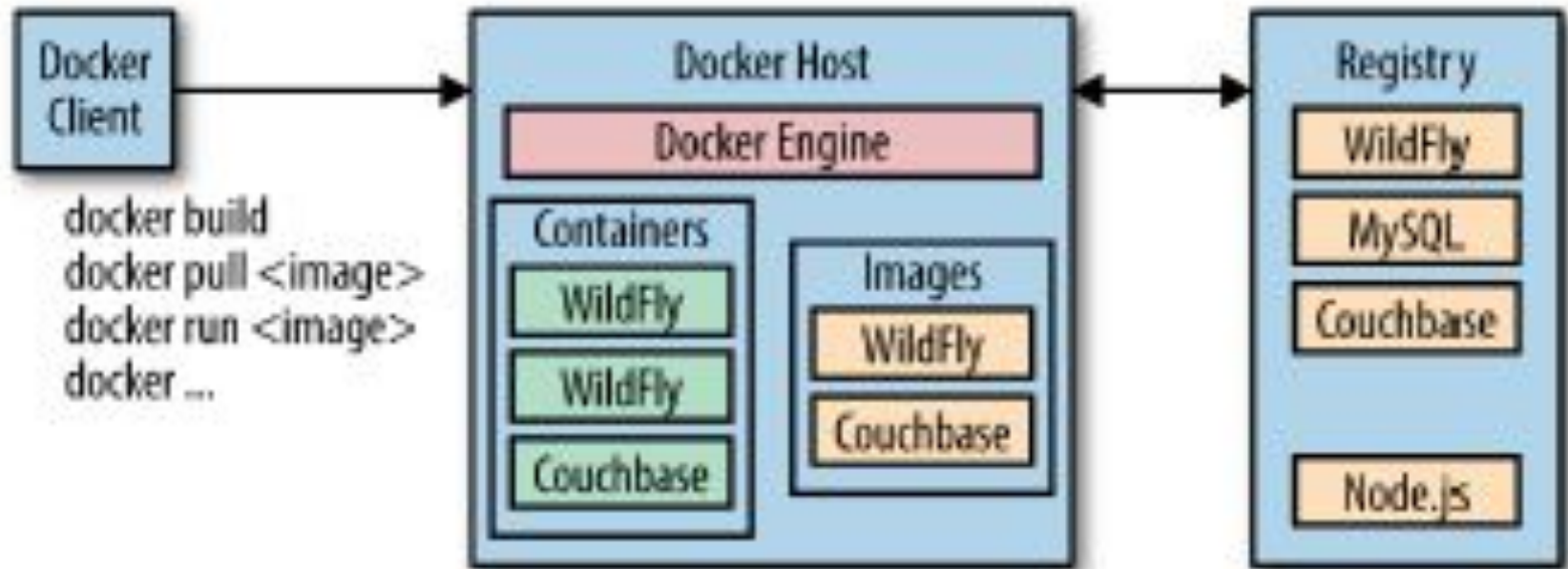
For more examples and ideas, visit:
https://docs.docker.com/userguide/
```

Docker workflow: 1st and 2nd run



- The container is started
 - Again: Not a VM, but an encapsulated process on the host system
- Processes are started within the container
 - These are specified in the definition of the image
- The container runs as long as a defined process is running
 - The container itself is then terminated

Docker in the big picture



- Provides a read-only environment
 - Analogy OOP: A class definition
- In contrast to VMWare images built from layers
 - Inheritance and association
- Due to the layering, individual images are therefore relatively slim
- Images and layers are managed in repositories
 - Meta information included
 - Versioning!
 - The Docker client installation manages a local repository

Example: A Java image (<https://microbadger.com/images/java>)

Tags	openjdk-8u111 openjdk-8u111-jdk openjdk-8 openjdk-8-jdk latest jdk 8u111 8u111-jdk 8 8-jdk
Created	January 17, 2017 at 01:52 AM
ID	d11c3799fa6a
Download Size	232.1 MB
Labels	No labels
Layers	14

107.2 MB

buildpack-deps

[scm](#) [jessie-scm](#)

What's this? -

49.0 MB

ADD file:89ecb642d662ee7edbb868340551106d51336c7e589... +

CMD ["/bin/bash"]

17.7 MB

RUN apt-get update && apt-get install -y --no-instal... +

40.5 MB

RUN apt-get update && apt-get install -y --no-instal... +

579.2 kB

RUN apt-get update && apt-get install -y --no-install-recommends b... +

214 bytes

RUN echo 'deb http://deb.debian.org/debian jessie-backports main' > ... +

ENV LANG=C.UTF-8

242 bytes

RUN { echo '#!/bin/sh'; echo 'set -e'; echo; echo 'dirname "... +

ENV JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64

ENV JAVA_VERSION=8u111

ENV JAVA_DEBIAN_VERSION=8u111-b14-2-bpo8+1

32 bytes

ENV CA_CERTIFICATES_JAVA_VERSION=20140324

124.1 MB

RUN set -x && apt-get update && apt-get install -y openjdk-8-jdk... +

282.3 kB

RUN /var/lib/dpkg/info/ca-certificates-java.postinst configure

- One instance of an image
 - Analogy to OOP: An object
 - Identification via a name or a technical key
 - A hash
- A container has a life cycle
 - Controlled via Docker commands
 - `create`
 - `run`
 - `start`
 - `stop`
 - `delete`
- In contrast to images, containers can have a state
 - Container environment
 - Internal resources
 - File system of the container

- Containers are completely encapsulated
 - Interaction only via Docker commands
 - Reading out the Docker log
 - Inspect
 - Executing a shell in the container
- Use of external volumes
 - Part of the container's file system provided by the host
- Network communication
 - Containers can provide network sockets
 - These are mapped to real sockets of the host
- Container linking
 - Direct communication between the containers
 - Various gradations
 - Joint network
 - Common file layer
 - Shared memory

- <https://docs.docker.com/engine/reference/commandline/docker/#/related-commands>
- **Elementary commands**
 - **Create container**
 - `sudo docker create <<image name>>:<<version number>>`
 - **Delete container**
 - `sudo docker rm <<containerId>>`
 - **Automatically delete Docker containers after execution**
 - `sudo docker run -rm <<image name>>`
 - **Show all containers**
 - `sudo docker ps -a`
 - **Start container**
 - `sudo docker start <<containerId>>`
 - **Create container, start and associate your own output console (not in the terminal session in which you are working)**
 - `sudo docker run --detach <<image name>>`
 - **Display images**
 - `sudo docker images`

- Creating an empty directory
 - All directories and subdirectories become part of the image
 - excludes in `.dockerignore`
- Creating a Docker file
 - Consisting of commands
 - FROM
 - COPY
 - ENV
 - RUN
 - EXPOSE

- The Dockerfile itself is a simple text document
 - Reference at <https://docs.docker.com/engine/reference/builder/>
 - Example

```
FROM openjdk
CMD ["java", "-version"]
```
- Creating the image
 - `docker build -tag <name>`

← → ↺

Sicher | https://docs.docker.com/engine/reference/builder/

 docker docs

Guides

Product Manuals

Glossary

Reference

Samples

Dockerfile reference

Cloud API

Cloud stack file reference

Compose file reference

Compose (docker-compose) CLI

Daemon CLI reference (dockerd)

Docker ID Accounts API

Engine API

Engine (docker) CLI

Machine (docker-machine) CLI

Registry API

Trusted Registry API

Trusted Registry CLI

Dockerfile reference

Estimated reading time: 60 minutes

Dockerfile reference

Docker can build images automatically by reading the instructions from a `Dockerfile`. A `Dockerfile` is a text document that contains all the commands a user could call on the command line to assemble an image. Using `docker build` users can create an automated build that executes several command-line instructions in succession.

This page describes the commands you can use in a `Dockerfile`. When you are done reading this page, refer to the `Dockerfile` [Best Practices](#) for a tip-oriented guide.

Usage

The `docker build` command builds an image from a `Dockerfile` and a `context`. The build's context is the files at a specified location `PATH` or `URL`. The `PATH` is a directory on your local filesystem. The `URL` is a Git repository location.

A context is processed recursively. So, a `PATH` includes any subdirectories and the `URL` includes the repository and its submodules. A simple build command that uses the current directory as context:

```
$ docker build .
Sending build context to Docker daemon 6.51 MB
...
```

The build is run by the Docker daemon, not by the CLI. The first thing a build process does is send the entire context (recursively) to the daemon. In most cases, it's best to start with an empty directory as context and keep your Dockerfile in that directory. Add only the

Encapsulated services with Docker

